

CrewAI

Complete Study Guide

Multi-Agent AI Systems

AI & Data Science Learning Journey
Phase 2 — AI Frameworks

Topics Covered: Agents | Tasks | Crews | Tools | Memory | Sequential & Hierarchical Processes

Section 1: What is CrewAI?

The Big Picture

Imagine you are a manager at a company. You don't do everything yourself — you hire specialists: a researcher who finds information, a writer who drafts content, and a reviewer who checks quality. Each person has a role, a goal, and a background. You assign them tasks, they work together, and the output of one feeds into the next.

CrewAI is exactly this — but with AI agents. CrewAI lets you build a team (called a "Crew") of AI agents that collaborate to complete complex, multi-step tasks. Each agent is an expert with a defined personality and purpose.

Why CrewAI? — Context vs Other Tools

Tool	Best Used For
LlamaIndex	RAG — one agent querying documents for answers
LangChain	Building single-agent pipelines and chains manually
CrewAI	Multiple AI agents with different roles collaborating on complex tasks
LangGraph	Graph-based workflows where you need fine-grained control over state

Key Philosophy

- CrewAI gives each agent a personality (role + goal + backstory) — this affects how it thinks and responds
- Agents can use tools (web search, file reading, APIs) to interact with the real world
- The output of one agent automatically becomes the input for the next
- CrewAI is built on top of LangChain under the hood but hides the complexity
- You focus on WHAT you want done — CrewAI handles HOW the agents coordinate

Note

CrewAI is used in production by companies for automated research pipelines, content generation, data analysis workflows, and customer support automation.

Section 2: The 4 Core Building Blocks

2.1 — Agent

An Agent is an AI worker. Think of it as a specialist employee with a defined identity.

Property	What It Means
role	Job title — e.g. "Senior Researcher", "Content Writer"
goal	What this agent is trying to achieve — its primary objective
backstory	Personality and background — affects tone and reasoning style
llm	The language model that powers this agent (GPT-4, Llama3, etc.)
tools	List of tools the agent can use (web search, file reader, etc.)
memory	Whether the agent remembers previous interactions (True/False)
verbose	Whether to print detailed logs of the agent's thinking (True/False)
allow_delegation	Whether this agent can delegate tasks to other agents

Agent Code Example

```
from crewai import Agent
from langchain_groq import ChatGroq

llm = ChatGroq(model='llama3-8b-8192', temperature=0.3)

researcher = Agent(
    role='Senior Researcher',
    goal='Find accurate, detailed information on any given topic',
    backstory='''You are an expert researcher with 10 years of experience.
    You always find reliable sources and summarize key facts clearly.''' ,
    tools=[SerperDevTool()], # can search the web
    llm=llm,
    verbose=True
)
```

Key Insight

The backstory is not just cosmetic — it actually influences how the LLM responds. A researcher with 'expert in academic papers' backstory will write differently than one with 'casual blogger' backstory.

2.2 — Task

A Task is a specific piece of work assigned to an agent. It defines WHAT needs to be done and WHAT the output should look like.

Property	What It Means
description	Detailed instructions for what needs to be done. Can contain {placeholders}.
expected_output	Tells the agent exactly what format the result should be in
agent	Which agent is responsible for this task
context	List of other tasks whose outputs this task should receive as input
tools	Override the agent's tools for this specific task (optional)
async_execution	Run this task asynchronously (parallel) — default is False

Task Code Example

```
from crewai import Task

research_task = Task(
    description='''Research the following tech topic: {topic}
    Find: what it is, why it matters, how it works,
    and 2-3 real world use cases.''',
    expected_output='''A structured summary with these sections:
    - What it is (2-3 sentences)
    - Why it matters (2-3 sentences)
    - How it works (3-4 sentences)
    - Real world examples (2-3 bullet points)''',
    agent=researcher # the researcher agent does this task
)
```

Tip

Use {placeholders} in task descriptions. When you call `crew.kickoff(inputs={'topic': 'RAG'})`, the placeholder gets replaced with the actual value. Same crew can handle any topic!

2.3 — Crew

A Crew is the team — it holds all agents and tasks together and defines how they collaborate.

Property	What It Means
agents	List of all agents in the crew
tasks	List of all tasks (defines the order of execution)
process	How agents work: Process.sequential or Process.hierarchical
verbose	Show detailed step-by-step logs of the entire crew's work
memory	Enable shared memory across the entire crew
max_rpm	Limit API calls per minute to avoid rate limiting
manager_llm	The LLM for the manager agent (hierarchical process only)

Crew Code Example

```
from crewai import Crew, Process

crew = Crew(
    agents=[researcher, writer, reviewer],
    tasks=[research_task, writing_task, review_task],
    process=Process.sequential, # tasks run one by one in order
    verbose=True
)

# Run the crew!
result = crew.kickoff(inputs={'topic': 'Retrieval Augmented Generation'})
print(result)
```

2.4 — Tools

Tools give agents real-world abilities beyond just generating text. Without tools, an agent can only think. With tools, it can act.

Tool	What It Does
SerperDevTool	Search Google via Serper API — gives agent internet access

ScrapeWebsiteTool	Read the full content of any web page URL
FileReadTool	Read content from a local file on disk
DirectoryReadTool	List all files in a folder
PDFSearchTool	Search inside a PDF document using RAG
YoutubeVideoSearchTool	Search YouTube for relevant videos
CodeInterpreterTool	Execute Python code and return results
BaseTool (custom)	Create your own tool with any functionality

Custom Tool Example

```
from crewai.tools import BaseTool

class MyCalculatorTool(BaseTool):
    name: str = 'Calculator'
    description: str = 'Use this to perform math calculations. Input: math expression'

    def _run(self, expression: str) -> str:
        result = eval(expression)    # e.g. '2 + 2 * 10'
        return str(result)

# Assign to an agent
analyst = Agent(
    role='Data Analyst',
    goal='Analyze numbers accurately',
    backstory='You are great at crunching numbers.',
    tools=[MyCalculatorTool()],
    llm=llm
)
```

Section 3: Process Types

3.1 — Sequential Process (Most Common)

In sequential process, tasks run one by one in the order you defined them. The output of Task 1 automatically becomes available as context for Task 2, and so on.

Task 1 (Researcher) --> Output --> Task 2 (Writer) --> Output --> Task 3 (Reviewer) --> Final Result

```
crew = Crew(
    agents=[researcher, writer, reviewer],
    tasks=[research_task, writing_task, review_task],
    process=Process.sequential, # default mode
    verbose=True
)
```

Auto Context

In sequential mode, you do NOT need to set context=[] on tasks. Each task automatically receives the previous task's output. Only use context=[] when a task needs output from a NON-adjacent earlier task.

3.2 — Hierarchical Process (Advanced)

In hierarchical process, a manager agent acts as a coordinator. It reads all the tasks and decides which agent should do what, in what order. This is more flexible but requires a manager LLM.

Manager Agent --> decides order and delegation --> Worker Agents --> Final Result

```
crew = Crew(
    agents=[researcher, writer, reviewer],
    tasks=[research_task, writing_task, review_task],
    process=Process.hierarchical,
    manager_llm=ChatGroq(model='llama3-70b-8192'), # smarter model for manager
    verbose=True
)
```

Process	When to Use
Sequential	Tasks have a clear order; output of one feeds directly into next; most common use case
Hierarchical	Tasks are flexible; need a coordinator; complex workflows where order isn't fixed

Section 4: Context Passing and Memory

4.1 — Context Between Tasks

By default in sequential process, each task receives only the immediately previous task's output. But what if Task 3 needs output from BOTH Task 1 and Task 2? Use the context parameter.

```
# Task 3 receives output from BOTH Task 1 and Task 2
review_task = Task(
    description='Review the research AND the article. Check accuracy and
    quality.',
    expected_output='A list of corrections and a quality score out of 10.',
    agent=reviewer,
    context=[research_task, writing_task] # explicitly receive both outputs
)
```

4.2 — Memory in CrewAI

Memory allows agents to remember things within and across runs. There are 3 types of memory:

Memory Type	Scope	How It Works
Short-term memory	Within one crew run	Agent remembers earlier parts of the same task session
Long-term memory	Across multiple crew runs	Stored in a database; agent learns from past runs
Entity memory	Within one run	Agent specifically tracks and remembers people, places, concepts mentioned

```
# Enable memory on an agent
researcher = Agent(
    role='Researcher',
    goal='...',
    backstory='...',
    memory=True, # agent remembers previous interactions
    llm=llm
)
```

```
# Enable crew-level memory (shared across all agents)
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    memory=True,    # shared crew memory
    process=Process.sequential
)
```

Section 5: Full Project — Tech Topic Explainer

This is the complete project built during the learning session. A 3-agent crew that researches any tech topic, simplifies it for beginners, and creates quiz questions.

Project Structure

- my_crewai_project/tech_explainer/main.py

Agent 1 — Tech Researcher

```
researcher = Agent(  
    role='Tech Researcher',  
    goal='Find clear and accurate information about any given tech topic',  
    backstory='''You are a senior AI researcher with deep knowledge of  
machine learning, data science, and software engineering.  
You always find the most important facts and present them clearly.'''  
    llm=llm,  
    verbose=True  
)
```

Agent 2 — Tech Simplifier

```
simplifier = Agent(  
    role='Tech Explainer',  
    goal='Explain complex tech concepts in the simplest possible way',  
    backstory='''You are an expert at breaking down complex topics for beginners.  
You use simple language, real-world analogies, and short sentences.  
You explain things as if talking to a 10-year-old.'''  
    llm=llm,  
    verbose=True  
)
```

Agent 3 — Quiz Maker

```
quiz_maker = Agent(  
    role='Quiz Creator',  
    goal='Create engaging quiz questions to test understanding of a topic',  
    backstory='''You are an experienced educator who creates clear,  
fair quiz questions. Your questions test real understanding,  
not just memorization. You always provide 4 options and mark the correct  
answer.'''  
)
```

```
    llm=llm,  
    verbose=True  
)
```

3 Tasks

```
research_task = Task(  
    description='''Research the following tech topic thoroughly: {topic}  
    Find: what it is, why it matters, how it works (simply),  
    and 2-3 real world use cases.'''  
    expected_output='A structured summary with 4 sections.'  
    agent=researcher  
)  
  
simplify_task = Task(  
    description='''Take the research provided and rewrite it as a  
    simple explanation for a complete beginner.  
    Use an analogy from real life. Keep it under 150 words.'''  
    expected_output='A beginner-friendly explanation under 150 words.'  
    agent=simplifier  
)  
  
quiz_task = Task(  
    description='''Based on the simplified explanation provided,  
    create exactly 3 multiple choice questions with 4 options (A,B,C,D).  
    Mark the correct answer for each.'''  
    expected_output='Exactly 3 quiz questions in Q/A format.'  
    agent=quiz_maker  
)
```

Crew + Run

```
import os  
from crewai import Agent, Task, Crew, Process  
from langchain_groq import ChatGroq  
  
os.environ['GROQ_API_KEY'] = 'your_key_here'  
  
llm = ChatGroq(model='llama3-8b-8192', temperature=0.3)  
  
# ... (agents and tasks defined above)  
  
crew = Crew(  
    agents=[researcher, simplifier, quiz_maker],  
    tasks=[research_task, simplify_task, quiz_task],  
    process=Process.sequential,
```

```
        verbose=True
    )

    result = crew.kickoff(inputs={'topic': 'Retrieval Augmented Generation (RAG) in
    AI'})
    print(result)
```

Result

Running this produces: a detailed research summary, a simple beginner explanation with analogy, and 3 multiple-choice quiz questions — all automated!

Section 6: Installation and Setup

Install Required Packages

```
pip install crewai crewai-tools langchain-groq
```

Groq Setup (Free LLM)

1. Go to <https://console.groq.com> and sign up
2. Click 'API Keys' in left sidebar
3. Click 'Create API Key' and copy it
4. Set it as environment variable or in your Python file

```
# Option 1: Set in Python file (easiest for learning)
import os
os.environ['GROQ_API_KEY'] = 'your_key_here'

# Option 2: Windows CMD
# set GROQ_API_KEY=your_key_here

# Option 3: PowerShell
# $env:GROQ_API_KEY='your_key_here'
```

Available Groq Models

Model	Speed	Best For
llama3-8b-8192	Fastest	Quick tasks, testing, learning
llama3-70b-8192	Smartest	Complex reasoning, production use
mixtral-8x7b-32768	Long context	Large documents, extended conversations

Test Groq Connection

```
from langchain_groq import ChatGroq

llm = ChatGroq(model='llama3-8b-8192', temperature=0.3)
response = llm.invoke('Say hello and tell me what model you are.')
print(response.content)
```

```
# Expected output:  
# Hello! I am Llama 3, an AI assistant made by Meta...
```

Section 7: Quick Reference — Cheat Sheet

Complete Import Statement

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool, ScrapeWebsiteTool, FileReadTool
from crewai.tools import BaseTool
from langchain_groq import ChatGroq
import os
```

Minimal Crew Template

```
import os
from crewai import Agent, Task, Crew, Process
from langchain_groq import ChatGroq

os.environ['GROQ_API_KEY'] = 'your_key_here'
llm = ChatGroq(model='llama3-8b-8192', temperature=0.3)

# 1. Create agents
agent1 = Agent(role='...', goal='...', backstory='...', llm=llm)
agent2 = Agent(role='...', goal='...', backstory='...', llm=llm)

# 2. Create tasks
task1 = Task(description='... {input}', expected_output='...', agent=agent1)
task2 = Task(description='...', expected_output='...', agent=agent2)

# 3. Create crew
crew = Crew(agents=[agent1, agent2], tasks=[task1, task2],
process=Process.sequential)

# 4. Run!
result = crew.kickoff(inputs={'input': 'your topic here'})
print(result)
```

Key Concepts at a Glance

Concept	One-Line Summary
Agent	AI worker with role, goal, backstory, tools, and an LLM
Task	A specific job for an agent with a description and expected output

Crew	The team — holds agents + tasks + defines how they collaborate
Tool	Gives agents real-world abilities (search, read files, run code)
Process.sequential	Tasks run in order; each gets previous task's output automatically
Process.hierarchical	A manager agent coordinates who does what and in what order
context=[]	Explicitly pass specific earlier task outputs to a task
memory=True	Agent remembers previous interactions
{placeholder}	Dynamic input in task description — filled in at kickoff
crew.kickoff(inputs={})	Starts the crew with the given input values

Section 8: Interview Questions & Answers

Review these Q&A cards to test and reinforce your understanding.

Q: What is CrewAI and what problem does it solve?

A: CrewAI is a Python framework for building multi-agent AI systems where multiple AI agents with different roles collaborate to complete complex tasks. It solves the problem of tasks that are too complex for a single AI agent — by breaking them into specialized roles (researcher, writer, reviewer) and letting agents work as a team, just like human employees in a company.

Q: What are the 4 core building blocks of CrewAI?

A: 1) Agent — the AI worker with role, goal, backstory, tools, and LLM. 2) Task — the specific job to be done, with a description and expected output format. 3) Crew — the team that holds agents and tasks together and defines the process. 4) Tool — capabilities that give agents real-world abilities like web search or file reading.

Q: What is the difference between an Agent and a Task in CrewAI?

A: An Agent is WHO does the work — it defines the AI worker's personality (role, goal, backstory), capabilities (tools), and brain (LLM). A Task is WHAT needs to be done — it defines the specific job, the instructions (description), and what a good result looks like (expected_output). You assign a task to an agent.

Q: What is the role of 'backstory' in an Agent? Is it just decorative?

A: No, backstory is not just decorative. It directly influences how the LLM responds. A researcher with backstory 'expert in academic papers with 10 years experience' will write differently than one with 'casual blogger' backstory. The backstory sets the personality, tone, and reasoning style of the agent. It is one of the most powerful ways to control agent behavior.

Q: What is the difference between Process.sequential and Process.hierarchical?

A: Sequential: Tasks run one by one in the defined order. Output of Task 1 automatically flows to Task 2, and so on. Simple, predictable, most common. Hierarchical: A manager agent coordinates the work, deciding which agent does what and in what order. More flexible but requires a manager_llm to be set. Use sequential for most cases, hierarchical for complex workflows where task order isn't fixed.

Q: In sequential process, how does Task 2 receive the output of Task 1?

A: Automatically — you don't need to write any code for this. In `Process.sequential`, CrewAI automatically passes the output of each task to the next task in the list. If you need a task to receive outputs from multiple earlier tasks (not just the immediately previous one), you use the `context=[]` parameter to explicitly specify which tasks to pull from.

Q: What is the context parameter in a Task and when do you use it?

A: The context parameter lets you specify which earlier task outputs a task should receive. By default in sequential mode, a task only gets the immediately previous task's output. If Task 3 needs outputs from BOTH Task 1 and Task 2, you set: `context=[task1, task2]`. This is useful when a reviewer agent needs both the original research AND the written article to do a proper review.

Q: How do you create a custom tool in CrewAI?

A: You inherit from `BaseTool` and implement the `_run()` method. You must define: `name` (string identifier), `description` (tells the LLM when to use this tool), and `_run(input)` method (the actual logic). Example: `class CalculatorTool(BaseTool): name='Calculator'; description='For math'; def _run(self, expr): return str(eval(expr))`. Then pass an instance to the agent's tools list.

Q: What is the purpose of {placeholders} in task descriptions?

A: Placeholders like `{topic}` or `{input}` make tasks dynamic and reusable. Instead of hardcoding 'Research Python' in the description, you write 'Research {topic}'. When running the crew, you pass: `crew.kickoff(inputs={'topic': 'Python'})`. This means the same crew and same tasks can handle any topic without changing the code — making the crew reusable and flexible.

Q: What types of memory does CrewAI support?

A: Three types: 1) Short-term memory — agent remembers within the current crew run session. 2) Long-term memory — stored in a database, agent learns from previous crew runs across sessions. 3) Entity memory — agent specifically tracks and remembers entities (people, places, concepts) mentioned during the run. Enable with `memory=True` on the Agent or Crew object.

Q: How is CrewAI different from LangChain?

A: LangChain is a low-level framework where you manually chain components together. CrewAI is built on top of LangChain but adds a high-level role-based abstraction. In LangChain you write: `prompt | model | parser`. In CrewAI you define agents with personalities and let them collaborate naturally. CrewAI is specifically designed for multi-agent collaboration scenarios, while LangChain is more general purpose.

Q: What LLMs can you use with CrewAI and how do you configure them?

A: CrewAI supports any LangChain-compatible LLM. Popular options: OpenAI (ChatOpenAI), Groq (ChatGroq — free tier), Ollama (local LLMs), Anthropic Claude, Azure OpenAI, Google Gemini. You create the LLM object once and pass it to each agent's `llm` parameter. Each agent can use a different LLM if needed — e.g. a fast cheap model for simple tasks and a powerful model for complex reasoning.

Q: What is the output of `crew.kickoff()` and how do you use it?

A: `crew.kickoff()` returns a `CrewOutput` object containing the final result — which is the output of the LAST task in the sequence. You can print it directly (`print(result)`) or access `result.raw` for the raw string output. If you need outputs from intermediate tasks, you can access them via the task objects after the crew runs. The final output represents the culmination of all agents' work.

Q: When would you use CrewAI vs LlamaIndex in a real project?

A: Use LlamaIndex when: you need to query documents/databases with RAG, the task is primarily information retrieval, one agent asking questions is sufficient. Use CrewAI when: the task requires multiple steps with different expertise, you need agents to specialize (researcher + writer + reviewer), the workflow is complex enough that one agent would struggle, or when you want modular, maintainable AI pipelines. In practice, you can COMBINE them — a CrewAI agent can use a LlamaIndex query engine as its tool!

Section 9: Quick Summary Table

Component	Analogy	Key Properties
Agent	Employee / Specialist	role, goal, backstory, tools, llm, memory
Task	Job Assignment	description, expected_output, agent, context
Crew	Team / Company	agents, tasks, process, verbose, memory
Tool	Equipment / Software	name, description, _run() method
Process.sequential	Assembly Line	Task 1 -> Task 2 -> Task 3, linear
Process.hierarchical	Management Structure	Manager decides order and delegation
context=[]	CC on email	Share specific earlier task outputs
{placeholder}	Mail merge field	Dynamic inputs filled at kickoff time

CrewAI in One Sentence

"Define AI workers with personalities, give them jobs, connect them as a team, and run with one command."